Уровни в игре

Уровни в игре

Вы уже знакомы (или познакомитесь в ближайшее время) с `алгоритмом Дейкстры` для поиска кратчайшего пути в графе. Его можно использовать для маршрутов персонажей. Но это не идеальный вариант.

Дейкстра находит кратчайший путь от старта до *всех* вершин, расширяясь равномерно во все стороны. Для игры это расточительно: нам нужен путь до одной конкретной цели, а алгоритм тратит время на противоположную сторону карты. Но он простой и базовый.

А также поднимает вопрос. А как связана игра с графом и алгоритмом Дейкстры? Тут все зависит от вас и вашей реализации.

Графы и уровни

Предположим, вы вывели на экран следующее изображение уровня:

- Как представить уровень в программе?
- Что вообще здесь может являться координатой в понимании игры? Отдельный пиксель изображения? Отдельный квадрат?

Современные мониторы имеют разрешение 1920x1080 и более, следовательно, ваш уровень с большой вероятностью будет занимать гораздо больше места. Вы ведь не хотите, чтобы человек приглядывался, или, чтобы ваши уровни были настолько маленькими, что влезали бы на один экран? Так что возьмем 5000x5000. И это только одно изображение, которое используется как ландшафт в вашей игре. А поверх будет изображение персонажа, всяких объектов, возможно, что рядом будут другие уровни игры.

- Все это сразу хранить в памяти?
- А как понимать, что различные объекты столкнулись?
- А как отслеживать, что наш персонаж стоит в грязи и должен двигаться медленно? Или он подошел к двери, и мы должны вывести оповещение "Нажмите E, чтобы открыть дверь"?

В общем, не затрагивая даже техническую часть, выглядит так, что работать напрямую с изображением в качестве уровня не удобно. Нам чего-то не хватает.

Это даже звучит довольно странно, под “уровнем” мы понимаем какую-то игровую сущность - у неё есть название, какие-то правила, особенности, позиция по отношению к другим уровням и т.д. - а под изображением - просто набор пикселей.

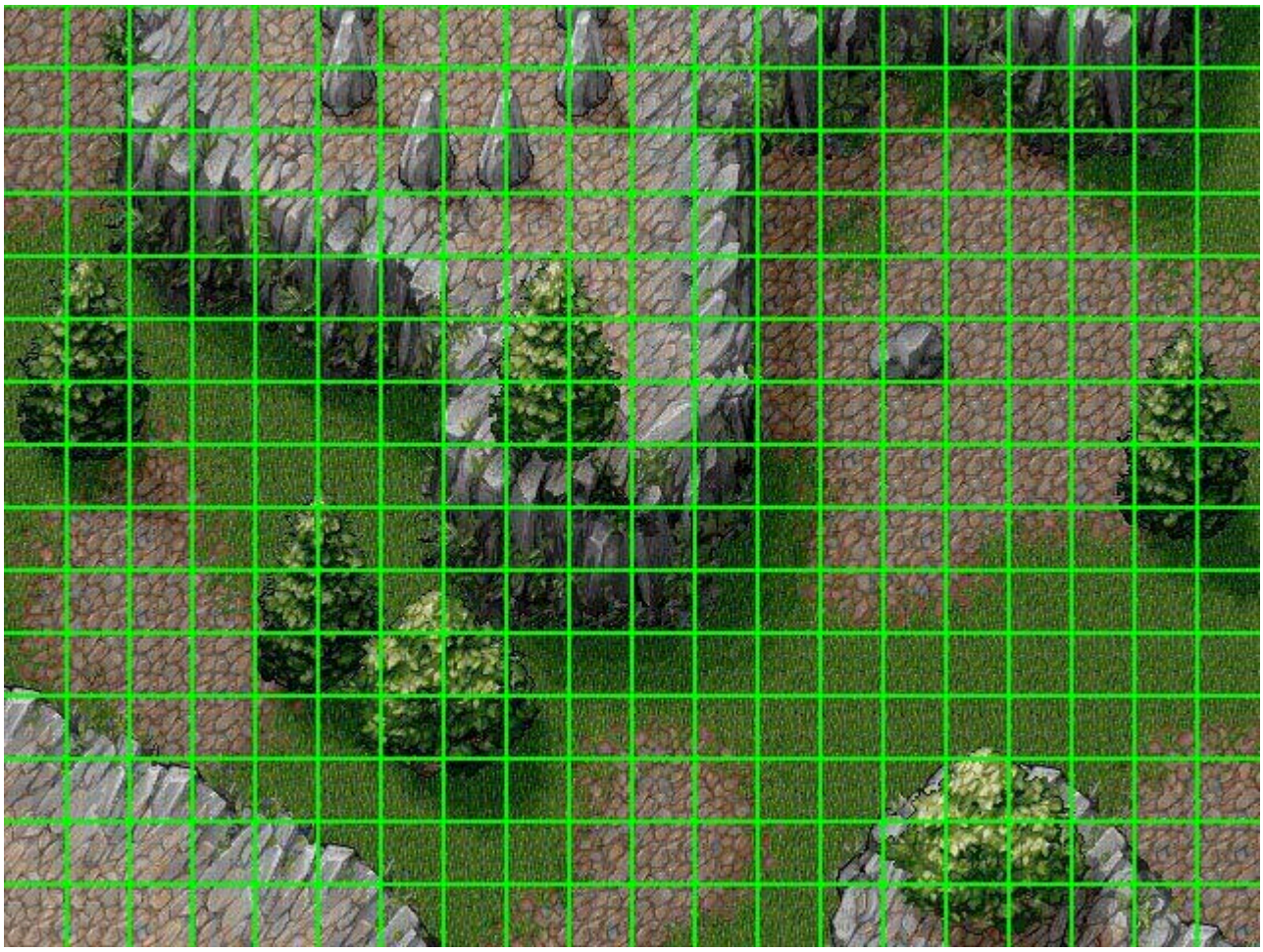
Следовательно, стоит выделить отдельную сущность в вашей игре - уровень. Или ландшафт, который уже будет принадлежать уровню.
И разбить изображение выше на какие-то отдельные составляющие.

Либо, создать некоторое упрощенное представление.

Зачем? Чтобы было удобнее работать с ним.

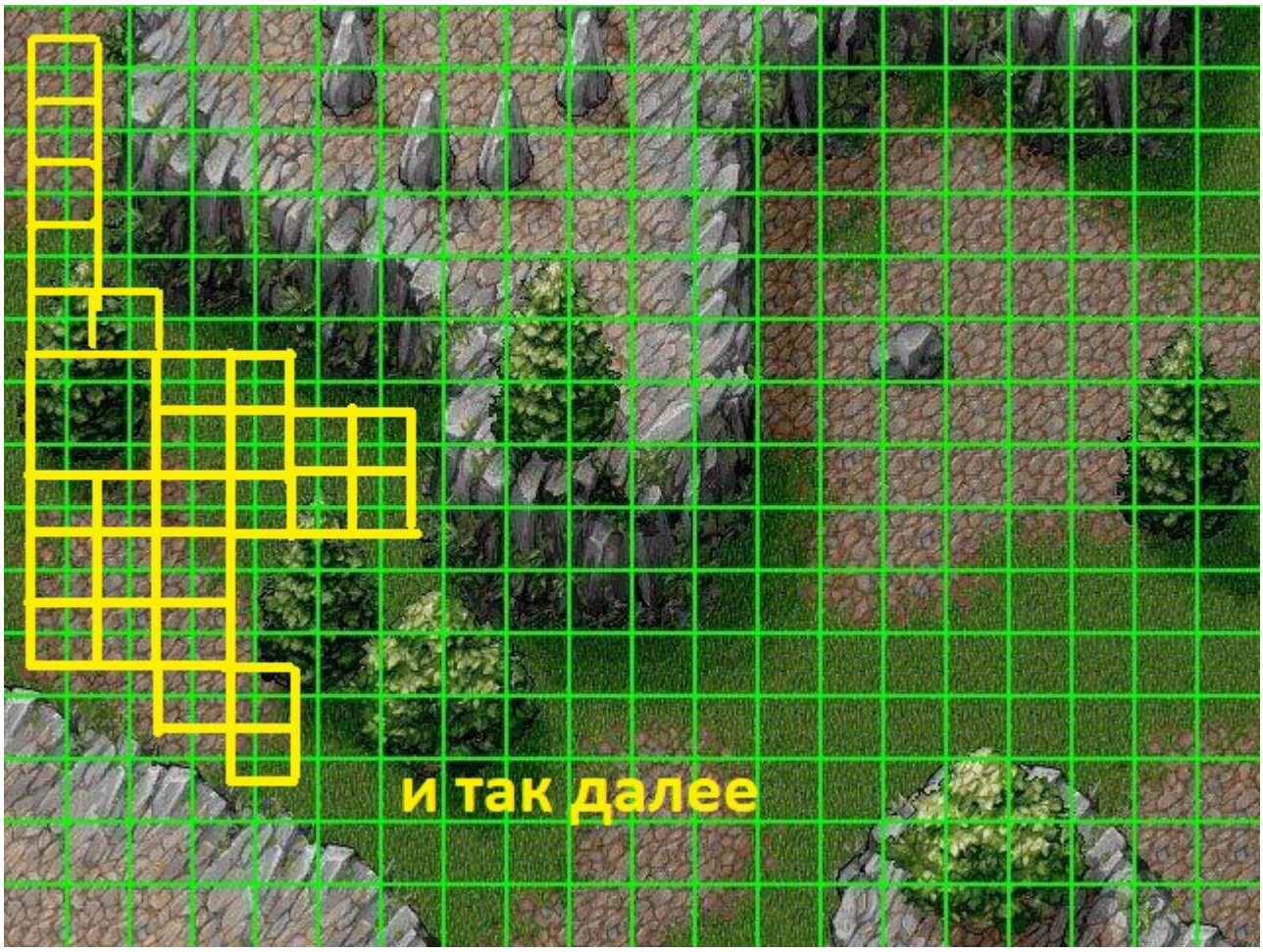
Тайлы

И теперь в первом варианте мы можем сказать так: наш ландшафт состоит из тайлов (“плиток”), каждая из которых ответственна за конкретную координату на уровне, имеет свою текстуру (изображение), имеет какие-то характеристики, влияющие на персонажа (например, тайл песка может иметь характеристику “вязкости”, снижающую скорость):



Более того, каждый тайл имеет свои размеры и координаты. Т.е. мы сгруппировали пиксели, абстрагировались таким образом.

Ок. Но граф тут все равно не появился. На самом деле, что такое граф? Вершины и связи (ребра) между ними. Возьмем наши тайлы за вершины и проведем между ними связи, которые показывают, что можно пройти от одного тайла к другому:



Обратите внимание на стену. Вершины, разделенные стеной, не соединены ребрами напрямую.

Вообще, хорошо бы еще автоматически генерировать граф, основываясь на данных об уровне - местоположении объектов, ландшафте и т.д.

С другой стороны, можно и не генерировать отдельный граф из ваших тайлов, а модифицировать алгоритмы так, чтобы они работали напрямую с тайловой сеткой.

К тому же, как хранить эти тайлы? Самый простой вариант, конечно, в виде матрицы. Причем, если мы захотим добавить третье измерение к нашему ландшафту, это можно сделать, добавив несколько матриц, каждая из которых будет представлять из себя отдельный слой на уровне.

Итоги по тайлам

Разбиение на тайлы (да и любое подобное разбиение) дает нам:

- **Упрощение разработки**: редактировать и создавать уровни становится легче.
- **Оптимизацию**: загрузка и рендеринг отдельных тайлов эффективнее за счет того, что мы можем использовать одни и те же тайлы в разных частях уровня, маскируя их одинаковость левел-дизайном.
- **Масштабируемость**: разбиение позволяет отсекать то, что мы не видим, не загружать то, что не используем, позволяя работать в целом с большим количеством объектов и размерами уровней.

Единственный ли это вариант? Нет, конечно. Но теперь у нас `есть граф`, с которым мы можем работать. И к нему можно применять алгоритмы, работающие с графами.

A* (A-Star)

Проблема: Дейкстра ищет везде

В игре нам обычно нужен путь до одной конкретной точки: враг бежит к игроку, а не ко всем тайлам на карте.

Дейкстра расширяет поиск равномерно во все стороны - как круговая волна. Если цель справа, он всё равно будет исследовать тайлы слева, сверху, снизу. Это лишняя работа.

Идея: подсказать направление

А что, если подсказать алгоритму, где находится цель? Именно это делает **A***. По сути, это Дейкстра с одной добавкой.

В алгоритме Дейкстры мы выбираем следующую вершину по **g** - реальной стоимости пути от старта. A* добавляет ещё одно слагаемое:

- **g** - реальная стоимость пути от старта до тайла (как у Дейкстры)
- **h** - **эвристика**, грубая оценка "а сколько ещё осталось до цели?"

Итоговый приоритет: **f = g + h**. Всё. Это вся разница.

Грубо говоря: Дейкстра спрашивает "откуда я пришёл?", а A* спрашивает "откуда я пришёл И куда мне идти?". Тайлы с меньшим f рассматриваются первыми - т.е. алгоритм предпочитает те, которые и близки к старту, и близки к цели.

Обратите внимание на GIF: волна расширяется *в сторону цели*, а не равномерно. Это и есть эффект эвристики.

Что за эвристика?

Эвристика - это наша "догадка" о расстоянии до цели. Мы не знаем точный путь (иначе зачем искать?), но можем прикинуть.

Для тайловой сетки без диагоналей подходит **манхэттенское расстояние** - сколько шагов по горизонтали и вертикали:

Python C#

```python
def heuristic(a, b):
    return abs(a[0] - b[0]) + abs(a[1] - b[1])
```

Если разрешены диагонали и диагональный шаг стоит столько же, сколько прямой - расстояние Чебышёва: `max(|dx|, |dy|)`. Если диагональ стоит √2 (что чаще встречается в играх) - октильное расстояние: `max(|dx|, |dy|) + (√2 − 1) · min(|dx|, |dy|)`.

A* гарантирует оптимальный путь, если эвристика **допустимая** (admissible) - т.е. никогда не переоценивает реальное расстояние. Манхэттенское расстояние на сетке без диагоналей допустимо. А вот если поставить `h = 0`, A* превращается ровно в Дейкстру.

Ключевое отличие в коде

Если у вас уже написан Дейкстра на тайловой сетке, переделать его в A* - одна строчка. Вместо `g` в качестве приоритета, пишем `g + h`:

Python C#

```python
# Dijkstra: cost so far only
priority = g[neighbor]

# A*: cost so far + estimated remaining
priority = g[neighbor] + heuristic(neighbor, goal)
```

Всё остальное - очередь с приоритетом, обход соседей, восстановление пути - остаётся таким же. Т.е. если вы реализовали Дейкстру на занятиях, вы уже почти реализовали A*.

Практические советы

Не пересчитывайте путь каждый кадр. Достаточно раз в 0.5–1 секунду, или когда игрок перешёл на другой тайл. Враг идёт по уже найденному пути, а пересчитывает только при необходимости.

Взвешенные тайлы. Если песок замедляет, а дорога ускоряет - стоимость перехода на соседа не 1, а `tile_cost[nx][ny]`. Алгоритм сам найдёт маршрут в обход болота, если обходной путь дешевле.

Итоги

A* - прямой потомок алгоритма Дейкстры из курса дискретной математики. Это пример **информированного поиска** (informed search) - когда алгоритм использует дополнительные знания о задаче для ускорения. В курсе ИИ вы встретите это как часть теории поиска в пространстве состояний.

Что дальше?

Тема	О чём
Коллизии	Пересечение тайлов - как считать столкновения на сетке
Пространственные структуры	QuadTree, Uniform Grid - как хранить и искать объекты эффективнее
Камера	Что видит игрок и что нужно отрисовывать
ИИ врагов	A* в контексте FSM: Patrol -> Chase с обходом препятствий
Управляющие силы	Когда нужно плавное движение вместо дискретных тайлов
Процедурная генерация	Генерируем уровни - A* валидирует проходимость
Алгоритмы	Обзор всех алгоритмических тем

Powered by Wiki.js